Objective	Traditional method	Akka method
Scaling	Uses a mix of threads, shared mutable state in a database (create, insert, update, delete), and web service RPC calls	Actors send and receive messages. No shared mutable state. Log of immutable events.
Providing interactive information	Polls for current information	Event-driven: pushes when the event occurs
Scaling out on the network	Synchronous RPC, blocking I/O	Asynchronous messaging, nonblocking I/O
Handling failures	Handles all exceptions; continues only if everything works	Lets the application crash. Isolates the failure, and continues without the failing parts.

Imagine that you have plans to conquer the world with a state-of-the-art chat application that will revolutionize the online collaboration space. It's focused on business users to help teams easily find each other and work together. You probably have many ideas about how this interactive application can connect to project management tools and integrate with existing communication services.

In good lean startup spirit, you start with a minimal viable product (MVP) of the chat application to learn as much as possible from your prospective users about what they need. If this ever takes off, you could potentially have millions of users (who doesn't chat or work together in teams?). But two forces can slow your progress to a grinding halt: